Normalizing Flows: Change of Variables & Boltzmann Generators

Lecture 15

Prof. Young Woo Choi

Department of Physics, Sogang University

PHYG004 / PHY5006 — 2026 Spring

6/5/2026

Generative models block: VAE (L14) → **Normalizing Flows (L15)** → Flow Matching (L16) → Diffusion (L17)

Recap: where we are in the generative block

Lecture	Method	$\log p_\theta(x)$	Sampling
L14	VAE	lower bound (ELBO)	fast (1 pass)
L15	**Normalizing Flow**	**exact**	**fast (1 pass)**
L16	Flow Matching (CNF)	exact (ODE-based)	fast (ODE solve)
L17	Diffusion (DDPM)	bound	slow (T steps)

The key difference that makes normalizing flows tractable: the generative map $f_\theta : z \mapsto x$ is **deterministic and invertible**, so the integral over z disappears.

Today: derive the change-of-variables formula, build a RealNVP flow in JAX, train it on 2D data with exact log-likelihoods, then use it as a Boltzmann generator.

The change-of-variables formula

Let $z \in \mathbb{R}^d$ be a **base** random variable with known density $p_Z(z)$, and let $f : \mathbb{R}^d \to \mathbb{R}^d$ be a **smooth, invertible** (diffeomorphic) map. Define $x = f(z)$.

The density of x is:

$$\boxed{p_X(x) \;=\; p_Z\big(f^{-1}(x)\big)\;\left|\det J_{f^{-1}}(x)\right|}$$

where $J_{f^{-1}}(x) = \frac{\partial f^{-1}}{\partial x}$ is the Jacobian of the **inverse** map.

Equivalently, in terms of the forward map f (using the inverse function theorem):

$$p_X(x) \;=\; p_Z\big(f^{-1}(x)\big)\;\left|\det J_f(z)\right|^{-1}, \quad z = f^{-1}(x).$$

Why the formula requires invertibility

The change-of-variables formula holds **only when f is a bijection** (one-to-one and onto).

What goes wrong for non-monotone f?

Consider a scalar non-monotone $f : \mathbb{R} \to \mathbb{R}$ (e.g., $f(z) = z^2$). Then a single value $x = f(z)$ has **two pre-images** $z_+ = \sqrt{x}$ and $z_- = -\sqrt{x}$.

The single-branch formula $p_X(x) = p_Z(f^{-1}(x))\,|f'(f^{-1}(x))|^{-1}$ is **no longer correct**.

The true density must sum over **all pre-image branches**:

$$p_X(x) \;=\; \sum_{k:\, f(z_k)=x} \frac{p_Z(z_k)}{|f'(z_k)|}$$

Log-likelihood of a flow

Taking the log of the change-of-variables formula:

$$\log p_X(x) \;=\; \log p_Z\big(f^{-1}(x)\big) \;+\; \log \left|\det J_{f^{-1}}(x)\right|.$$

For a **composed** flow $f = f_K \circ f_{K-1} \circ \cdots \circ f_1$, the log-determinant telescopes:

$$\log p_X(x) \;=\; \log p_Z(z_0) \;+\; \sum_{k=1}^{K} \log \left|\det J_{f_k^{-1}}(x_k)\right|, \quad z_0 = f^{-1}(x).$$

No ELBO gap. This is an **exact** log-likelihood — not a lower bound. The price: f must be a diffeomorphism with a **tractable Jacobian determinant**.

> Designing architectures that achieve expressiveness while keeping $\left|\det J_f\right|$ cheap to compute is the core engineering challenge of normalizing flows.

The Jacobian determinant bottleneck

A naive $d \times d$ Jacobian has $O(d^2)$ entries and its determinant costs $O(d^3)$.

For $d = 784$ (MNIST), this is $784^3 \approx 5 \times 10^8$ operations — per sample, per step. Hopeless.

Three strategies to make $\left|\det J_f\right|$ cheap:

Triangular J

$\det J = \prod_i J_{ii}$

$O(d)$ cost.

NICE, RealNVP, Glow use **coupling layers** to enforce this.

Autoregressive J

Lower/upper triangular via autoregressive factorisation.

MAF, IAF, NAF.

Continuous (CNF)

Replace discrete layers with an ODE. Hutchinson trace estimator gives $O(d)$ stochastic approximation.

Flow Matching (L16) avoids even the ODE during training.

Affine coupling layers (NICE / RealNVP)

Split $x \in \mathbb{R}^d$ into two halves: $x = (x_A, x_B)$, $|A| + |B| = d$.

Forward (generation): $z \mapsto x$:

$$x_A = z_A, \qquad x_B = z_B \odot \exp\big(s_\theta(z_A)\big) \; + \; t_\theta(z_A).$$

Inverse (inference): $x \mapsto z$:

$$z_A = x_A, \qquad z_B = \big(x_B - t_\theta(x_A)\big) \odot \exp\big(-s_\theta(x_A)\big).$$

The Jacobian is **triangular**: $\partial x_A / \partial z_B = 0$ and $\partial x_B / \partial z_B = \mathrm{diag}(\exp(s_\theta(z_A)))$.

$$\log|\det J| \; = \; \sum_j s_\theta(z_A)_j \qquad O(d).$$

RealNVP: stacking coupling layers

A single coupling layer only transforms half the variables. Alternate which half is active:

$z \sim p_Z$ → Coupling (mask A) → Coupling (mask B) → $\cdots$ → $x \sim p_X$

Training objective (maximum likelihood, exact):

$$\mathcal{L}(\theta) \;=\; -\frac{1}{N}\sum_{i=1}^{N} \log p_X(x_i;\theta) \;=\; -\frac{1}{N}\sum_{i=1}^{N}\left[\log p_Z(z_i) + \sum_{k} \log\left|\det J_{f_k}(z_{k-1})\right|\right].$$

No ELBO, no variational gap. Both generation ($z \to x$) and density evaluation ($x \to z$) are $O(K)$ forward passes.

Variants: **Glow** adds 1×1 invertible convolutions for channel mixing; **neural spline flows** replace affine with piecewise-rational-quadratic transforms for greater expressiveness.

2D demo: two-moons target

Train a 4-layer RealNVP on the two-moons distribution. Both forward and inverse directions can be visualized.

Base space $z \sim \mathcal{N}(0, I)$

The flow warps the standard Gaussian into a crescent-shaped distribution. Contour lines of $p_Z(z)$ are pulled into the two-moon shape.

Data space $x \sim p_X$

After training, `flow.sample(1000)` draws exact independent samples. The log-likelihood on held-out data converges to the true value — not a bound.

Key check: train NLL should decrease monotonically. Unlike VAE training, there is no reconstruction/KL tug-of-war — just a single exact objective.

From density estimation to Boltzmann sampling

So far: we trained on **data** from p_{data}. But what if we know p analytically (up to a constant) but cannot sample it?

Boltzmann distribution:

$$p(x) = \frac{1}{Z} e^{-U(x)/k_B T}, \qquad Z = \int e^{-U(x)/k_B T}\, dx.$$

We know $U(x)$; we do not know Z; MCMC gets trapped.

Boltzmann generator idea (Noé *et al.*, Science 2019):

1. Parametrize a flow $f_\theta : z \mapsto x$, $z \sim \mathcal{N}(0, I)$.
2. Train to minimise the **reverse KL** (no data needed):

Forward vs reverse KL — why it matters for sampling

Forward KL $D_{\mathrm{KL}}(p \,\|\, q_\theta)$

$$\mathbb{E}_{x\sim p}\left[\log \frac{p(x)}{q_\theta(x)}\right]$$

- Penalised where $p > 0$ but $q_\theta \approx 0$
- **Mode-covering**: model must cover all modes
- Used in MLE training **with data** (L14 VAE, this lecture MLE variant)

Reverse KL $D_{\mathrm{KL}}(q_\theta \,\|\, p)$

$$\mathbb{E}_{x\sim q_\theta}\left[\log \frac{q_\theta(x)}{p(x)}\right]$$

- Penalised where $q_\theta > 0$ but $p \approx 0$
- **Mode-seeking**: model may concentrate on one mode
- Used in **Boltzmann generators** (data-free; energy known)

Key: for Boltzmann generators we sample from q_θ (the flow) and evaluate $\log p \propto -U/k_BT$ (known). No MCMC. No data. Just energy evaluations.

Boltzmann generator: double-well warm-up

Before tackling real molecules, build intuition with the 1D double-well $U(x) = (x^2 - 1)^2$.

MCMC at low T: the chain gets trapped in one well. A flow trained via reverse KL:

- learns both wells with correct relative weight
- generates **independent** samples (no autocorrelation)
- gives exact importance weights for reweighting to any T'

Importance reweighting. Flow samples are from q_θ, not exactly from p. Unbiased estimator of $\langle A \rangle_p$:

$$\langle A \rangle_p \;\approx\; \frac{\sum_i A(x_i)\, w(x_i)}{\sum_i w(x_i)}, \qquad w(x_i) = \frac{e^{-U(x_i)/k_B T}}{q_\theta(x_i)}.$$

If $q_\theta \approx p$, weights are nearly uniform — effective sample size is high.

Boltzmann generator: alanine dipeptide

The canonical benchmark for Boltzmann generators in the molecular setting (Noé *et al.* 2019).

System: alanine dipeptide in vacuum. Dihedral angles (ϕ, ψ) are the slow degrees of freedom; the free-energy landscape has well-separated basins.

Challenge: MCMC at room temperature rarely crosses the ϕ-barrier; long runs are needed to get ergodic statistics.

Flow setup:

- Input: Cartesian coordinates of all atoms (or internal coordinates)
- Base: Gaussian
- Training: reverse KL with force-field energy U_{ff}
- Evaluation: (ϕ, ψ) Ramachandran plot from flow samples vs. long MCMC

Comparison with VAE (L14)

	VAE (L14)	**Normalizing Flow (L15)**
$\log p_\theta(x)$	lower bound (ELBO)	**exact**
Latent	stochastic encoder	deterministic inverse
Architecture constraint	decoder only needs to be smooth	f must be **bijective**
Training signal	reconstruction + KL	exact NLL or reverse KL
Boltzmann sampling	possible (reverse KL on decoder) but approximate	exact (reverse KL, no gap)
Expressiveness per parameter	high (decoder unconstrained)	lower (bijection constraint)

NICE — the first coupling flow (2014)

NICE (Non-linear Independent Components Estimation, Dinh *et al.* 2014) introduced the additive coupling layer:

$$x_A = z_A, \qquad x_B = z_B + t_\theta(z_A).$$

The Jacobian determinant is **exactly 1** (volume-preserving). To allow volume changes, NICE adds a final diagonal **scaling layer** $x_i = s_i z_i$ (learned per-dimension scalars).

RealNVP (Dinh *et al.* 2016) generalised to the affine coupling layer (seen earlier), allowing non-volume-preserving transformations layer-by-layer. This dramatically increases expressiveness for the same number of layers.

Glow (Kingma & Dhariwal 2018) further added 1×1 invertible convolutions between coupling layers, enabling arbitrary permutation of channels (as opposed to fixed checkerboard/channel masks).

Neural spline flows

Affine coupling layers are limited: they map Gaussians to distributions that look "affinely warped" in each dimension. **Neural spline flows** (Durkan *et al.* 2019) replace the affine transform with a piecewise rational-quadratic spline:

$$x_B = \mathrm{RQS}\big(z_B;\, \theta(z_A)\big),$$

where RQS is a monotone rational-quadratic function with K knots (parameters output by a network from z_A).

Properties:

- Still triangular Jacobian → $O(d)$ log-det
- K knots per dimension → much more expressive per layer
- Smooth (C^2) unlike piecewise-linear alternatives
- Default choice in modern flow libraries (normflows, zuko, nflows)

Hands-on: what you will build today

Open `handson.ipynb`. The session has four parts:

1. **Change-of-variables warm-up** (scalar, 1D)
   - Verify the formula numerically for a monotone f
   - See what happens for non-monotone f (sum over pre-images)
2. **RealNVP on 2D toy data** (two-moons, checkerboard)
   - Implement coupling layers and the log-det Jacobian
   - Train with exact MLE; visualize the learned density
3. **Boltzmann generator: 1D double-well**
   - Train via reverse KL (energy-based, no data)
   - Compare with MCMC; compute importance weights

Looking ahead: Flow Matching (L16)

The bottleneck of normalizing flows: the bijection constraint limits architecture choices and makes high-dimensional data hard (images, proteins require many stacked layers).

Continuous normalizing flows (CNFs) parametrize the flow as an ODE:

$$\frac{dx}{dt} = v_\theta(x, t), \quad x(0) \sim p_Z, \quad x(1) \sim p_X.$$

The instantaneous change-of-variables formula (Chen *et al.* 2018):

$$\frac{d \log p(x(t))}{dt} = -\mathrm{tr}\left(\frac{\partial v_\theta}{\partial x}\right).$$

This removes the bijection constraint — **any** smooth velocity field works. But training requires solving an ODE backward through time (expensive).

Looking further: SBI / NPE (L19)

The flow machinery from today extends to **conditional** density estimation.

A **conditional normalizing flow** $q_\phi(z \mid x_{\text{obs}})$ maps a parameter vector z through a flow whose coupling-layer parameters depend on the observation x_{obs}.

Neural Posterior Estimation (NPE, L19):

1. Simulator: $z \sim p(z)$, $x \sim p(x \mid z)$.
2. Train a conditional flow to approximate $p(z \mid x)$ over many (z, x) pairs.
3. At test time: given a new x_{obs}, a single forward pass through the flow gives $q_\phi(z \mid x_{\text{obs}}) \approx p(z \mid x_{\text{obs}})$.

This is **amortized inference**: the same trained flow works for any x_{obs} without re-running MCMC.

Summary

Core ideas

- Change-of-variables formula: $p_X = p_Z(f^{-1})\,|\det J_{f^{-1}}|$
- Requires **bijection** — non-monotone f needs sum over all pre-images
- Affine coupling layers: triangular J, $O(d)$ log-det
- RealNVP: stack alternating coupling layers

Boltzmann generators

- Reverse KL training: use energy $U(x)$, no data
- Independent samples at inference — no MCMC autocorrelation
- Importance reweighting for unbiased estimators
- Benchmark: alanine dipeptide (ϕ, ψ) landscape

Where flows sit in the generative landscape:

Property	Flow	VAE	Flow Matching (L16)

References

Normalizing flows

- Dinh, Krueger & Bengio, *NICE: Non-linear Independent Components Estimation*, ICLR Workshop 2015. arXiv:1410.8516
- Dinh, Sohl-Dickstein & Bengio, *Density Estimation using Real-valued Non-Volume Preserving Transformations* (RealNVP), ICLR 2017. arXiv:1605.08803
- Kingma & Dhariwal, *Glow: Generative Flow with Invertible 1×1 Convolutions*, NeurIPS 2018. arXiv:1807.03039
- Durkan *et al.*, *Neural Spline Flows*, NeurIPS 2019. arXiv:1906.04032
- Papamakarios *et al.*, *Normalizing Flows for Probabilistic Modeling and Inference*, JMLR 22 (2021). arXiv:1912.02762 *(comprehensive review)*

Boltzmann generators

- Noé *et al.*, *Boltzmann Generators: Sampling Equilibrium States of Many-Body Systems with Deep Learning*, Science 365, eaaw1147 (2019).

Questions?

Open `handson.ipynb` — let's build a flow.

6/5/2026